

Instructions for Using the Provided Data

Please download the data from [our server](#). Password: `rnm_2026`

The directory contains two zipped ROS 2 MCAP recordings. You can unpack them using for example [7-zip](#) on Windows or [zip](#) on Linux. ROS 2 uses [rosbag2 with the MCAP storage plugin](#) for recording and playing back messages in realtime. You can inspect the content of an MCAP file with the command `ros2 bag info -s mcap filename.mcap`. You can play it back with `ros2 bag play -s mcap filename.mcap`. You can visualize the Kinect topics with `rviz2` (Type `rviz2` in a terminal, on the left side click on Add → By topic and visualize the respective topics as an 'Image'). The MCAP files can also be inspected with tools such as [Rerun](#) and [Foxglove](#).

The calibration dataset was recorded with Kinect mounted on the Panda robot. A static checkerboard calibration pattern (40mm square size, 8x5) was placed in the view of the camera. Then, the robot moved to 40 different poses. You can use this dataset to test your implementation for both camera calibration and eye in hand calibration. For the final application you will collect these datasets yourselves with the real robot.

The files regarding the calibration are:

- `calibration.mcap`: a ROS 2 MCAP recording containing the following topics
 - `/k4a/rgb/image_raw`: RGB image from the Kinect
 - `/k4a/depth/image_raw`: Depth image from the Kinect
 - `/k4a/ir/image_raw`: Infrared image from the Kinect
 - `/franka_state_controller/joint_states_desired`: The joint positions of the Panda robot
- `pose.txt`: The 40 4x4 robot poses in row-major. So the first pose is

-0.0109058	-0.117606	0.993003	0.189151
-0.737594	0.671458	0.071424	-0.0438333
-0.675161	-0.731652	-0.094068	0.694449
0	0	0	1
- `joints.txt`: The 40 robot joint configurations; you can use this data to verify your forward kinematics
- `trk.txt`: The 40 marker poses (checkerboard marker in rgb frame) in the same format as the robot poses
- `calibration_results.txt`: The factory calibration parameters to check your implementation (uses the 8 parameter rational model for intrinsic calibration)
- `hand_eye_results.txt`: Reference results and estimated errors of calibration. Note that the used algorithm (QR24) additionally returns the transformation between checkerboard and base (robot-world-hand-eye-calibration problem $AX=YB$), while for the given task only the eye-in-hand calibration ($AX=XB$) is strictly necessary.

The second dataset was recorded with the robot moving the camera around the skeleton phantom. You can use this data for stitching the Kinect images to a combined scan and then doing the registration to a high resolution model.

The files regarding the scanning are:

- scanning.mcap: a ROS 2 MCAP recording containing the following topics
 - /k4a/rgb/image_raw: RGB image from the Kinect
 - /k4a/depth/image_raw: Depth image from the Kinect
 - /k4a/points2: pointcloud in the rgb camera reference frame (uses the factory calibration to create the point cloud)
 - /franka_state_controller/joint_states_desired: the joint positions of the robot. You can also use your forward kinematics node to get the robot's end effector pose.
- calibration_results.txt: factory calibration of the Kinect
- handeye_result.txt: corresponding eye-in-hand calibration matrices
- Skeleton_Target.stl: A high resolution scan of the skeleton with an indicated target.